Linux Common Analysis & Observability Tools



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Pseudo Filesystems



Pseudo Filesystems

- Some virtual filesystems are exposed by the kernel and provide a lot of information on the system.
- *procfs* contains information about processes and system information.
  - Mounted on `/proc`
  - Often parsed by tools to display raw data in a more user-friendly way.
- *sysfs* provides information about hardware/logical devices, association between devices and drivers.
  - Mounted on `/sys`
- *debugfs* exposes information related to debug.
  - Typically mounted on `/sys/kernel/debug/`
  - `mount -t debugfs none /sys/kernel/debug`



procfs

- *procfs* exposes information about processes and system (man 5 proc).
  - `/proc/cpuinfo` CPU information.
  - `/proc/meminfo` memory information (used, free, total, etc).
  - `/proc/sys/` contains system parameters that can be tuned. The list of parameters that can be modified is available at admin-guide/sysctl/index
  - `/proc/interrupts`: interrupt count per CPU for each interrupt in use
    - We also have one entry per interrupt in `/proc/irq` for specific configuration/status for each interrupt line
  - `/proc/<pid>/` process related information
    - `/proc/<pid>/status` process basic information
    - `/proc/<pid>/maps` process memory mappings
    - `/proc/<pid>/fd` file descriptors of the process
    - `/proc/<pid>/task` descriptors of threads belonging to the process
  - `/proc/self/` will refer to the process used to access the file
- A list of all available *procfs* file and their content is described at filesystems/proc and man 5 proc



sysfs

- *sysfs* filesystem exposes information about various kernel subsystems, hardware devices and association with drivers (man 5 sysfs).
- `/sys/bus` contains one directory per bus types
  - `/sys/bus/<bus>/drivers` shows all drivers attached to a bus
  - `/sys/bus/<bus>/devices` shows all devices sitting on a bus
  - Symlinks inside those directories allows inspecting the relationship between devices and drivers
- `/sys/class` contains a tree of class devices registered by drivers
- `/sys/kernel` contains interesting files for kernel debugging:
  - `irq` with information about interrupts (mapping, count, etc).
  - `tracing` for tracing control.
- admin-guide/abi-stable



debugfs

- *debugfs* is a simple RAM-based filesystem which exposes debugging information.
- Used by some subsystems (*clk*, *block*, *dma*, *gpio*, etc) to expose debugging information related to the internals.
- Usually mounted on `/sys/kernel/debug`
  - Dynamic debug features exposed through `/sys/kernel/debug/dynamic_debug` (also exposed in `proc`)
  - Clock tree exposed through `/sys/kernel/debug/clk/clk_summary`.



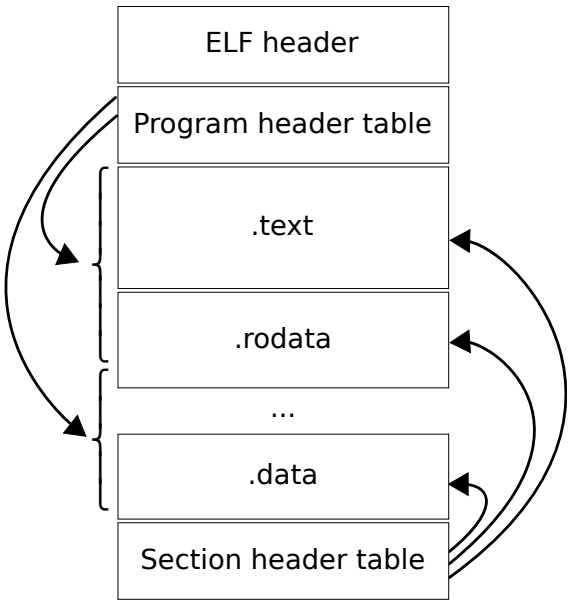
ELF file analysis



ELF files

Executable and **L**inkable **F**ormat

- File starting with a header which holds binary structures defining the file
- Collection of segments and sections that contain data
  - `.text` section: Code
  - `.data` section: Data
  - `.rodata` section: Read-only Data
  - `.debug_info` section: Contains debugging information
- Sections are part of a segment which can be loadable in memory
- Same format for all architectures supported by the kernel and also `vmlinux` format
  - Also used by a lot of other operating systems as the standard executable file format





binutils for ELF analysis

- The binutils are used to deal with binary files, either object files or executables.
  - Includes `ld`, `as` and other useful tools.
- *readelf* displays information about ELF files (header, section, segments, etc).
- *objdump* allows to display information and disassemble ELF files.
- *objcopy* can convert ELF files or extract/translate some parts of it.
- *nm* displays the list of symbols embedded in ELF files.
- *addr2line* finds the source code line/file pair from an address using an ELF file with debug information



binutils example (1/2)

- Finding the address of `ksys_read()` kernel function using *nm*:

```
$ nm vmlinux | grep ksys_read
c02c7040 T ksys_read
```

- Using *addr2line* to match a kernel OOPS address or a symbol name with source code:

```
$ addr2line -s -f -e vmlinux ffffffff8145a8b0
queue_wc_show
blk-sysfs.c:516
```



binutils example (2/2)

- Display an ELF header with *readelf*:

```
$ readelf -h binary
ELF Header:
Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
Class:                             ELF64
Data:                              2's complement, little endian
Version:                           1 (current)
OS/ABI:                            UNIX - System V
ABI Version:                       0
Type:                              DYN (Position-Independent Executable file)
Machine:                           Advanced Micro Devices X86-64
...
```

- Convert an ELF file to a flat binary file using *objcopy*:

```
$ objcopy -O binary file.elf file.bin
```



ldd

- In order to display the shared libraries used by an ELF binary, one can use *ldd* (Generally packaged with C library. See man 1 ldd).
- *ldd* will list all the libraries that were used at link time.
  - Libraries that are loaded at runtime using `dlopen()` are not displayed.

```
$ ldd /usr/bin/bash
linux-vdso.so.1 (0x00007ffdf3fc6000)
libreadline.so.8 => /usr/lib/libreadline.so.8 (0x00007fa2d2aef000)
libc.so.6 => /usr/lib/libc.so.6 (0x00007fa2d2905000)
libncursesw.so.6 => /usr/lib/libncursesw.so.6 (0x00007fa2d288e000)
/lib64/ld-linux-x86-64.so.2 => /usr/lib64/ld-linux-x86-64.so.2 (0x00007fa2d2c88000)
```



Monitoring tools



Monitoring Tools

- Lots of monitoring tools on Linux to allow monitoring various part of the system.
- Most of the time, these are CLI interactive programs.
  - Processes with *ps*, *top*, *htop*, etc
  - Memory with *free*, *vmstat*
  - Networking
- Almost all these tools rely on the *sysfs* or *procfs* filesystem to obtain the processes, memory and system information but will display them in a more human-readable way.
  - Networking tools use a netlink interface with the networking subsystem of the kernel.



Process and CPU monitoring tools



Processes with *ps*

- The *ps* command allows to display a snapshot of active processes and their associated information (man 1 ps)
  - Lists both user processes and kernel threads.
  - Displays PID, CPU usage, memory usage, uptime, etc.
  - Uses `/proc/<pid>/` directory to obtain process information.
  - Almost always present on embedded platforms (provided by *BusyBox*).
- By default, displays only the current user/current tty processes, but output is highly customizable:
  - `aux`/`-e`: show all processes
  - `-L`: show threads
  - `-p`: target a specific process
  - `-o`: select output columns to display
- Useful for scripting and parsing since its output is static.



ps example

- Display all processes in a friendly way:

```
$ ps aux
USER    PID %CPU %MEM    VSZ   RSS TTY STAT START  TIME COMMAND
root      1  0.0  0.0 168864 12800 ?   Ss   09:08  0:00 /sbin/init
root      2  0.0  0.0      0     0 ?   S    09:08  0:00 [kthreadd]
root      3  0.0  0.0      0     0 ?   I<   09:08  0:00 [rcu_gp]
root      4  0.0  0.0      0     0 ?   I<   09:08  0:00 [rcu_par_gp]
root      5  0.0  0.0      0     0 ?   I<   09:08  0:00 [netns]
[...]
root    914  0.0  0.0 396216 16220 ?   Ssl  09:08  0:04 /usr/libexec/udisks2/udisksd
avahi   929  0.0  0.0   8728   412 ?   S    09:08  0:00 avahi-daemon: chroot helper
root    956  0.0  0.1 260304 19024 ?   Ssl  09:08  0:02 /usr/sbin/NetworkManager [...]
root    960  0.0  0.0  17040  5704 ?   Ss   09:08  0:00 /sbin/wpa_supplicant -u [...]
root    962  0.0  0.0 317644 11896 ?   Ssl  09:08  0:00 /usr/sbin/ModemManager
vnstat  987  0.0  0.0   5516  3696 ?   Ss   09:08  0:00 /usr/sbin/vnstatd -n
```



Processes with *top*

- *top* command output information similar to *ps* but dynamic and interactive ().
  - Also almost always present on embedded platforms (provided by *BusyBox*)

```
$ top
top - 18:38:11 up  9:29,  1 user,  load average: 2.84, 2.74, 2.02
Tasks: 371 total,   1 running, 370 sleeping,   0 stopped,   0 zombie
%Cpu(s):  5.8 us,  2.1 sy,  0.0 ni, 77.4 id, 14.7 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  15947.6 total,   1476.9 free,   7685.7 used,   6784.9 buff/cache
MiB Swap:  15259.0 total,  15238.7 free,     20.2 used.   7742.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2988 cleger    20   0 5184816   1.2g 430244 S  26.7   7.9  60:24.27 firefox-esr
   4326 cleger    20   0   16.4g 208104  81504 S  26.7   1.3   9:27.33 code
    909 root     -51   0       0      0      0 S  13.3   0.0  15:12.15 irq/104-nvidia
  41704 cleger    20   0   38.4g 373744 116984 S  13.3   2.3  13:25.76 code
  91926 cleger    20   0 2514784 145360  95144 S  13.3   0.9   1:29.85 Web Content
```



mpstat

- *mpstat* displays Multiprocessor statistics (man 1 mpstat).
- Useful to detect unbalanced CPU workloads, bad IRQ affinity, etc.

```
$ mpstat -P ALL
Linux 6.0.0-1-amd64 (fixe)      19/10/2022      _x86_64_        (4 CPU)

17:02:50     CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
17:02:50     all    6,77    0,00    2,09   11,67    0,00    0,06    0,00    0,00    0,00   79,40
17:02:50       0    6,88    0,00    1,93    8,22    0,00    0,13    0,00    0,00    0,00   82,84
17:02:50       1    4,91    0,00    1,50    8,91    0,00    0,03    0,00    0,00    0,00   84,64
17:02:50       2    6,96    0,00    1,74    7,23    0,00    0,01    0,00    0,00    0,00   84,06
17:02:50       3    9,32    0,00    2,80   54,67    0,00    0,00    0,00    0,00    0,00   33,20
```